

Contents

Orbit-RS vs Snowflake: Comprehensive Competitive Analysis	1
Executive Summary	1
Table of Contents	1
1. Product Overview Comparison	1
2. Feature Comparison Matrix	2
3. Detailed Gap Analysis	5
4. Roadmap to Parity	8
5. Roadmap to Market Leadership	9
6. Competitive Positioning	11
7. Strategic Recommendations	12
Appendix A: Feature Implementation Estimates	12
Appendix B: TCO Analysis Example	13
Appendix C: Glossary	13

Orbit-RS vs Snowflake: Comprehensive Competitive Analysis

Version: 1.0 **Date:** December 13, 2025 **Status:** Confidential - Strategic Planning Document

Executive Summary

This whitepaper provides a comprehensive feature-by-feature comparison between Orbit-RS and Snowflake, identifying competitive gaps, parity opportunities, and strategic paths to market leadership. Orbit-RS, as an open-source, multi-protocol database with AI-native capabilities, presents unique advantages in flexibility, cost, and multi-modal data handling that differentiate it from Snowflake’s cloud data warehouse approach.

Key Findings

- **Orbit-RS Advantages:** Multi-protocol support, open-source, on-premise deployment, AI-native architecture, hardware acceleration
- **Parity Gaps:** Enterprise tooling, data sharing marketplace, governance features, cloud-native scaling
- **Market Leadership Opportunities:** AI/ML integration, real-time processing, edge computing, hybrid cloud

Table of Contents

- 1. [Product Overview Comparison](#)
- 2. [Feature Comparison Matrix](#)
- 3. [Detailed Gap Analysis](#)
- 4. [Roadmap to Parity](#)
- 5. [Roadmap to Market Leadership](#)
- 6. [Competitive Positioning](#)
- 7. [Strategic Recommendations](#)

1. Product Overview Comparison

Snowflake

- **Type:** Cloud-native data warehouse (SaaS)

- **Architecture:** Separated storage and compute, multi-cluster shared data
- **Deployment:** AWS, Azure, GCP only
- **Pricing:** Consumption-based (compute credits + storage)
- **Primary Use Cases:** Data warehousing, data lakes, data engineering, BI/analytics
- **Founded:** 2012, IPO 2020
- **Market Position:** Market leader in cloud data warehousing

Orbit-RS

- **Type:** Multi-protocol, multi-model database (Open Source + Commercial)
- **Architecture:** Unified storage with virtual actors, hardware-accelerated
- **Deployment:** On-premise, cloud, hybrid, edge
- **Pricing:** Open-source (BSD-3/MIT) + enterprise support/features
- **Primary Use Cases:** Multi-protocol data serving, real-time analytics, AI/ML workloads, edge computing
- **Status:** Pre-1.0 (v0.1.0 unreleased)
- **Market Position:** Emerging challenger with unique multi-protocol positioning

2. Feature Comparison Matrix

Legend

- **Full Feature** - Production-ready, feature-complete
- **Partial** - Available but limited or requires development
- **Not Available** - Not currently implemented
- **Roadmap** - Planned for future release

Feature Category	Feature	Snowflake	Orbit-RS	Gap Analysis
Data Storage	Columnar Storage			Orbit has columnar model but not optimized like Snowflake
	Semi-Structured (JSON/XML) Time Travel	(90 days)		Parity - both support JSON natively
	Zero-Copy Cloning Storage Compression	(auto)		Major gap - Snowflake's key feature Gap - needs implementation Orbit has LZ4/Snappy, needs auto-compression
	External Tables			Orbit has Iceberg, needs more formats
Query Engine	SQL Support	(ANSI SQL)		Parity - both ANSI SQL compliant
	Vectorized Execution			Parity - both use SIMD
	Query Optimization	(cost-based)	(AI-powered)	Different approaches, Orbit AI advantage

Feature Category	Feature	Snowflake	Orbit-RS	Gap Analysis
Scalability	Materialized Views			Gap - needs implementation
	Search			Orbit has FTS, needs bloom filters
	Optimization			Orbit has module
	Result Caching			caching, needs query cache
Data Sharing & Collaboration	Auto-Scaling			Snowflake fully automated, Orbit manual
	Multi-Cluster Warehouses			Orbit clustering exists, needs refinement
	Concurrency Scaling	(auto)		Orbit actor system helps, needs auto-scale
	Elastic Compute			Gap - Orbit needs cloud-native elasticity
Security & Governance	Secure Data Sharing			Major gap - Snowflake differentiator
	Data Marketplace			Major gap - requires ecosystem
	Data Exchange			Gap - needs data product layer
	Reader Accounts			Gap - needs implementation
Performance	End-to-End Encryption			Orbit has TLS, needs at-rest encryption
	Role-Based Access Control			Orbit has basic RBAC, needs refinement
	Column-Level Security			Gap - needs implementation
	Row-Level Security			Gap - needs implementation
Performance	Data Masking	(dynamic)		Gap - needs implementation
	Object Tagging			Gap - metadata management needed
	Audit			Orbit has basic logging, needs compliance
	Logging			Major gap - compliance certifications
Performance	SOC 2, HIPAA, PCI DSS			

Feature Category	Feature	Snowflake	Orbit-RS	Gap Analysis
Data Integration	Query Performance	(excellent)		Comparable - both use modern techniques
	Ingestion Speed	(high)		Parity - both handle high throughput
	GPU Acceleration			Orbit Advantage - Metal/CUDA/Vulkan
	SIMD Optimization			Parity - both use AVX-512
Multi-Protocol Support	Native Connectors	(400+)		Gap - Orbit has ~20 protocols
	Snowpipe (Streaming)			Orbit has CDC, needs Snowpipe-like
	External Functions			Orbit Advantage - Lua/Python/WASM
	Stored Procedures	(JavaScript/SQL)		Orbit Advantage - multi-language
AI/ML Capabilities	Tasks & Orchestration			Gap - Orbit needs workflow engine
	Streams (CDC)			Parity - both have CDC
	PostgreSQL Wire			Orbit Advantage
	MySQL Wire			Orbit Advantage
Developer Experience	Redis Protocol			Orbit Advantage
	Cassandra (CQL)			Orbit Advantage
	Neo4j (Bolt)			Orbit Advantage
	gRPC			Orbit Advantage
Developer Experience	Python UDFs			Parity
	ML Model Deployment	(Snowpark ML)		Gap - Orbit has inference, needs training
	Vector Similarity Search	(preview)		Parity - pgvector compatible
	AI-Powered Optimization			Orbit Advantage
Developer Experience	GPU-Accelerated ML			Orbit Advantage
	SQL Worksheets			Gap - needs web UI
	Notebooks	(Snowflake Notebooks)		Gap - needs Jupyter integration

Feature Category	Feature	Snowflake	Orbit-RS	Gap Analysis
Deployment	CLI Tools	(SnowSQL)		Parity - orbit-cli available
	REST API			Parity
	Python SDK			Parity
	JDBC/ODBC Drivers			Gap - needs certified drivers
	Cloud (AWS/Azure/GCP)	(only)		Orbit more flexible
	On-Premise			Orbit Advantage
	Hybrid Cloud			Orbit Advantage
Cost Model	Edge Computing			Orbit Advantage
	Kubernetes			Orbit Advantage - K8s operator
	Pricing Model	Consumption	Open-source + Support	Orbit Advantage - no lock-in
	Predictable Costs			Orbit Advantage - self-hosted
Data Models	No Vendor Lock-in			Orbit Advantage
	Relational Document	(JSON)		Parity
	Key-Value			Orbit has native document model
	Graph			Orbit Advantage - Redis protocol
	Time-Series			Orbit Advantage - Cypher support
	Vector	(preview)		Orbit Advantage - native TSDB
Observability				Parity/Orbit slight edge
	Query History			Gap - Orbit has basic, needs UI
	Query Profiling			Gap - needs visual profiler
	Resource Monitors			Gap - needs resource tracking UI
	Prometheus Metrics			Orbit Advantage

3. Detailed Gap Analysis

Critical Gaps (Must-Have for Parity)

3.1 Time Travel & Versioning **Snowflake Feature:** 90-day time travel, zero-copy cloning, fail-safe recovery

Current Orbit-RS: Not implemented

Gap Impact: CRITICAL - This is a killer feature for data warehousing - Users can query historical data without backups - Enables “undo” for data changes - Critical for compliance and auditing

Path to Parity: 1. Implement MVCC-based versioning at storage layer 2. Add snapshot isolation with configurable retention (default 7 days, up to 90) 3. Implement copy-on-write for zero-copy clones 4. Add SQL syntax: `SELECT * FROM table AT(TIMESTAMP => '2025-01-01 00:00:00')` 5. Implement UNDROP for tables/databases

Effort: 8-12 weeks, 2 engineers

3.2 Secure Data Sharing Snowflake Feature: Share live data with other Snowflake accounts without copying, monetizable via marketplace

Current Orbit-RS: Not implemented

Gap Impact: CRITICAL - Major differentiator for Snowflake - Enables data-as-a-product business models - No data movement or duplication - Fine-grained access control

Path to Parity: 1. Implement data share objects with read-only access 2. Create secure share protocol (authenticated, encrypted) 3. Build share catalog and discovery 4. Add metering and usage tracking 5. Implement cross-account/cross-deployment sharing

Effort: 16-20 weeks, 3 engineers

3.3 Enterprise Security & Governance Snowflake Features: - Column/row-level security - Dynamic data masking - Object tagging - SOC 2, HIPAA, PCI DSS compliance

Current Orbit-RS: Basic security, no fine-grained controls

Gap Impact: HIGH - Required for enterprise adoption - Banks, healthcare, finance require these features - Compliance is table stakes for large enterprises

Path to Parity: 1. **Column-Level Security** (4 weeks) - Implement column-level grants: `GRANT SELECT(col1, col2) ON table TO role` - Enforce at query planning stage

2. **Row-Level Security** (6 weeks)

- Implement row security policies: `CREATE ROW ACCESS POLICY`
- Automatic filter injection at query time

3. **Dynamic Data Masking** (4 weeks)

- Masking policies per column: `CREATE MASKING POLICY email_mask AS (val) RETURNS ...`
- Multiple masking functions (hash, partial, null, custom)

4. **Compliance Certifications** (24-36 weeks)

- SOC 2 Type II audit (\$50k-100k)
- HIPAA compliance review
- PCI DSS certification if handling payment data
- Document security controls, penetration testing

Effort: 38-50 weeks total, 2-3 engineers + compliance consultant

3.4 Auto-Scaling & Elastic Compute Snowflake Feature: Automatic warehouse scaling, multi-cluster warehouses, instant scale-up/down

Current Orbit-RS: Manual clustering, basic auto-scaling

Gap Impact: HIGH - Critical for cloud-native workloads - Users expect “infinite” scale - Pay-per-use requires automatic scaling - Concurrency management

Path to Parity: 1. **Cloud-Native Orchestration** (8 weeks) - Kubernetes HPA (Horizontal Pod Autoscaler) integration - Custom metrics for query queue depth, CPU, memory - Auto-scale compute nodes based on workload

2. **Multi-Cluster Compute** (12 weeks)
 - Implement warehouse abstraction (compute pool)
 - Route queries to least-loaded cluster
 - Automatic cluster provisioning/deprovisioning
3. **Serverless Mode** (16 weeks)
 - Pre-warmed compute pools
 - Sub-second cold start times
 - Pay-per-query pricing model

Effort: 36 weeks, 3-4 engineers

High-Priority Gaps (Important for Competitiveness)

3.5 Web-Based IDE & Notebooks **Snowflake Feature:** Snowsight UI with SQL worksheets, visualizations, Snowflake Notebooks (Jupyter-based)

Current Orbit-RS: CLI only, no web UI

Gap Impact: MEDIUM-HIGH - Analysts expect web-based tools

Path to Parity: 1. **SQL Worksheet** (12 weeks) - Web-based query editor (Monaco/CodeMirror) - Syntax highlighting, autocomplete - Result visualization (tables, charts) - Query history and saved queries

2. **Jupyter Integration** (8 weeks)
 - Native Orbit kernel for Jupyter
 - Magic commands (`%orbit`, `%sql`)
 - DataFrame integration (Polars/Pandas)
3. **Data Catalog UI** (8 weeks)
 - Browse databases, schemas, tables
 - View metadata, statistics, lineage
 - Search across data catalog

Effort: 28 weeks, 2-3 frontend engineers

3.6 Native Connectors & Integrations **Snowflake Feature:** 400+ pre-built connectors via Snowflake Partner Network

Current Orbit-RS: ~20 protocols, limited ETL connectors

Gap Impact: MEDIUM - Ecosystem lock-in effect

Path to Parity: 1. **Top 50 Connectors** (24 weeks) - Salesforce, SAP, Oracle, SQL Server - AWS S3, Azure Blob, GCS - Kafka, Kinesis, Pub/Sub - Tableau, Power BI, Looker - DBT, Airflow, Fivetran

2. **Connector SDK** (8 weeks)
 - Plugin architecture for custom connectors
 - Standard connector interface
 - Connector marketplace

Effort: 32 weeks, 2 engineers

3.7 Tasks & Orchestration **Snowflake Feature:** Scheduled tasks, task graphs (DAGs), error handling, notifications

Current Orbit-RS: No native orchestration

Gap Impact: MEDIUM - Users rely on external tools (Airflow), but native is preferred

Path to Parity: 1. **Task Scheduler** (8 weeks) - Cron-based scheduling: `CREATE TASK ... SCHEDULE = 'USING CRON 0 9 * * * UTC'` - Task dependencies (DAGs) - Error handling and retry logic

2. **Serverless Tasks** (6 weeks)

- Automatic compute provisioning for tasks
- Concurrent task execution
- Task monitoring and alerting

Effort: 14 weeks, 2 engineers

Strategic Gaps (Ecosystem & Market Position)

3.8 Data Marketplace & Ecosystem **Snowflake Feature:** Snowflake Marketplace with 2000+ data products, monetization

Current Orbit-RS: No marketplace

Gap Impact: MEDIUM (long-term strategic)

Path to Parity: 1. Build open data marketplace platform (24 weeks) 2. Onboard initial data providers (12 weeks) 3. Implement billing/metering for paid data (8 weeks)

Effort: 44 weeks, 3 engineers + BD team

4. Roadmap to Parity

Phase 1: Critical Features (6-9 months)

Goal: Achieve minimum viable parity for enterprise adoption

1. **Time Travel & Versioning** (Q1 2026)

- MVCC implementation
- 7-day time travel (default), 90-day option
- Zero-copy cloning
- UNDROP functionality

2. **Enterprise Security** (Q1-Q2 2026)

- Column-level security
- Row-level security
- Dynamic data masking
- Enhanced audit logging

3. **Auto-Scaling** (Q2 2026)

- Kubernetes-based auto-scaling
- Multi-cluster compute pools
- Automatic concurrency scaling

4. **Web IDE** (Q2 2026)

- SQL worksheet with visualization
- Query history and saved queries
- Basic data catalog UI

Deliverables: Orbit-RS v0.2.0 - “Enterprise Ready”

Phase 2: Competitive Features (9-18 months)

Goal: Match Snowflake’s core feature set

1. **Secure Data Sharing** (Q3 2026)
 - Share protocol implementation
 - Cross-deployment sharing
 - Usage metering
2. **Materialized Views** (Q3 2026)
 - Automatic refresh
 - Incremental maintenance
3. **Tasks & Orchestration** (Q3-Q4 2026)
 - Task scheduler with DAGs
 - Serverless task execution
4. **Native Connectors** (Q4 2026)
 - Top 25 source/destination connectors
 - Connector SDK and marketplace
5. **Jupyter Notebooks** (Q4 2026)
 - Native Orbit kernel
 - DataFrame integration
6. **Compliance Certifications** (Q4 2026 - Q1 2027)
 - SOC 2 Type II
 - HIPAA readiness
 - PCI DSS (if applicable)

Deliverables: Orbit-RS v0.3.0 - “Feature Parity”

Phase 3: Strategic Differentiation (18-36 months)

Goal: Achieve market leadership through unique capabilities

1. **Data Marketplace** (Q1-Q2 2027)
 - Open marketplace platform
 - Monetization and billing
2. **Advanced ML Features** (Q1-Q3 2027)
 - Distributed model training
 - AutoML pipelines
 - Model versioning and registry
3. **Edge Computing** (Q2-Q3 2027)
 - Edge deployment optimizations
 - Sync protocols for edge ↔ cloud
 - Lightweight edge runtime

Deliverables: Orbit-RS v1.0 - “Market Leader”

5. Roadmap to Market Leadership

Differentiation Strategy: “Beyond Data Warehousing”

Snowflake is a cloud data warehouse. Orbit-RS can be **the unified data platform** for modern applications.

Key Differentiators

5.1 Multi-Protocol Support (Existing Advantage) Market Positioning: “One Database, Every Protocol”

Features: - PostgreSQL, MySQL, Redis, Cassandra, Neo4j protocols - Real-time serving (Redis) + analytics (SQL) in one platform - Graph queries (Cypher) + relational (SQL) on same data

Go-to-Market: - Target: Applications that need both OLTP and OLAP - Use case: E-commerce (Redis cache + PostgreSQL analytics + Neo4j recommendations) - Pitch: “Replace 3 databases with one Orbit-RS deployment”

Investment: Minimal - already implemented, needs marketing

5.2 AI-Native Architecture (Existing Advantage) Market Positioning: “Database with Built-In Intelligence”

Features: - AI-powered query optimization - GPU-accelerated analytics - Native vector search - AutoML for predictive analytics - Automatic anomaly detection - Smart caching based on ML predictions

Go-to-Market: - Target: AI/ML teams frustrated with Snowpark ML limitations - Use case: Real-time ML inference + model training on same platform - Pitch: “10x faster ML workloads with GPU acceleration”

Investment: 8-12 months, 3-4 ML engineers

5.3 Hybrid Cloud & Edge (Unique Advantage) Market Positioning: “From Cloud to Edge, One Platform”

Features: - On-premise deployment - Kubernetes operator - Edge runtime (low-memory, ARM support) - Bidirectional sync (edge - cloud) - Smart tiering (hot/warm/cold/edge)

Go-to-Market: - Target: Retail, manufacturing, IoT with edge requirements - Use case: Store analytics at edge + centralized data warehouse - Pitch: “Analytics where your data lives - cloud, data center, or edge”

Investment: 12-18 months, 3 engineers

5.4 Open Source & Cost Efficiency Market Positioning: “Snowflake Performance, 1/10th the Cost”

Features: - Open-source core (BSD-3/MIT) - Self-hosted (no compute charges) - Commercial-grade support offerings - Managed service option (optional)

Go-to-Market: - Target: Cost-conscious enterprises, scale-ups - Use case: Migrating from Snowflake to cut costs 70-90% - Pitch: TCO analysis tools showing savings

Investment: Minimal - positioning and case studies

5.5 Real-Time Analytics (10x Improvement Opportunity) Market Positioning: “Millisecond Analytics, Not Minutes”

Features: - Streaming ingestion (CDC) - Sub-second query latency (vs Snowflake’s seconds) - Materialized views with <100ms refresh - Real-time dashboards - Streaming SQL (continuous queries)

Go-to-Market: - Target: Operational analytics, monitoring, real-time BI - Use case: Real-time fraud detection, live dashboards - Pitch: “Snowflake for data warehousing, Orbit for real-time analytics”

Investment: 16-24 months, 4-5 engineers

5.6 Developer-First UDFs (Existing Advantage) Market Positioning: “Write Functions in ANY Language”

Features: - Lua, Python, WASM UDFs - WASM with SIMD, multi-threading, streaming - Rust, Go, C++ UDF support - UDF marketplace - Containerized UDFs (Docker functions)

Go-to-Market: - Target: Developer-heavy organizations - Use case: Custom ML models, business logic in database - Pitch: “Bring your code to the data, not data to the code”

Investment: 6-12 months, 2 engineers

6. Competitive Positioning

Market Segmentation Strategy

Segment	Primary Competitor	Orbit-RS Positioning	Win Strategy
Enterprise Data Warehouse	Snowflake	“Open-source alternative with lower TCO”	Cost savings (70-90%), no vendor lock-in
Real-Time Analytics	Clickhouse, Druid	“Real-time + batch in one platform”	Unified architecture, simpler stack
Multi-Model Database	MongoDB, Neo4j	“One database, every data model”	Multi-protocol support, cost consolidation
Edge Computing	None (greenfield)	“Cloud to edge analytics platform”	Unique positioning, no direct competition
AI/ML Workloads	Databricks	“GPU-native database for ML”	Hardware acceleration, lower cost
Hybrid Cloud	Oracle, IBM Db2	“Modern hybrid data platform”	Cloud-native + on-premise flexibility

Competitive Messaging Framework

Against Snowflake When to Use Orbit-RS Instead: 1. Need on-premise or hybrid cloud deployment (compliance, data sovereignty) 2. Multi-protocol requirements (Redis + SQL in same database) 3. Cost-sensitive (self-hosted reduces costs 70-90%) 4. Real-time analytics (<100ms latency requirements) 5. Open-source preference (no vendor lock-in) 6. GPU acceleration for ML workloads

When Snowflake Wins: 1. Need zero operational overhead (fully managed) 2. Require mature data marketplace ecosystem 3. Need proven compliance certifications (SOC 2, HIPAA certified) 4. Want seamless multi-cloud without managing infrastructure 5. Require Snowflake-specific integrations (Snowpipe, Streams mature ecosystem)

Migration Path from Snowflake: 1. Dual-write pattern (write to both Snowflake and Orbit-RS) 2. Migrate read queries incrementally 3. Switch writes to Orbit-RS, replicate to Snowflake for transition period 4. Full cutover after validation 5. Estimated migration time: 3-6 months for typical deployment

7. Strategic Recommendations

Immediate Actions (Next 3 Months)

1. Feature Prioritization

- Focus on time travel (highest ROI, most requested)
- Implement column/row-level security (enterprise blocker)
- Build MVP web UI (usability requirement)

2. Competitive Intelligence

- Monitor Snowflake Summit announcements
- Track Snowflake pricing changes
- Identify customer pain points via community/forums

3. Go-to-Market

- Create TCO calculator (Snowflake vs Orbit-RS)
- Publish benchmark comparisons (query performance, cost)
- Target Snowflake customers on Hacker News, Reddit

Mid-Term Strategy (6-12 Months)

1. Product Development

- Achieve Phase 1 parity (time travel, security, auto-scaling)
- Launch managed cloud offering (Orbit Cloud)
- Build migration tools (Snowflake → Orbit-RS)

2. Market Positioning

- Position as “Snowflake alternative” in press/media
- Sponsor data engineering conferences
- Create certification program

3. Ecosystem Building

- Partner with DBT for native integration
- Integrate with Airbyte/Fivetran for data ingestion
- Build BI tool connectors (Tableau, Looker, Metabase)

Long-Term Vision (12-36 Months)

1. Market Leadership

- Become #1 open-source data warehouse
- Achieve 10,000+ production deployments
- Build thriving community (GitHub stars, contributors)

2. Enterprise Adoption

- Win 100+ enterprise customers
- Achieve SOC 2, HIPAA certifications
- Build professional services team

3. Strategic Differentiation

- Lead in real-time analytics category
- Dominate hybrid cloud use cases
- Become go-to platform for AI/ML workloads

Appendix A: Feature Implementation Estimates

Feature	Priority	Effort (weeks)	Engineers	Dependency
Time Travel (7-day)	P0	8	2	Storage layer MVCC
Zero-Copy Cloning	P0	4	1	Time travel
Column-Level Security	P0	4	1	Query planner

Feature	Priority	Effort (weeks)	Engineers	Dependency
Row-Level Security	P0	6	2	Query planner
Dynamic Data Masking	P1	4	1	Row-level security
Web SQL Worksheet	P0	8	2	None
Auto-Scaling (K8s)	P0	8	2	K8s operator
Multi-Cluster Compute	P1	12	3	Auto-scaling
Secure Data Sharing	P1	16	3	Time travel, security
Materialized Views	P1	8	2	None
Task Scheduler	P1	8	2	None
Jupyter Integration	P2	8	2	Python SDK
Native Connectors (Top 25)	P1	24	2	Connector SDK
Data Marketplace	P2	24	3	Data sharing
SOC 2 Certification	P0	24	1 + consultant	Security features

Total Effort for Parity: ~180 engineer-weeks (~36 months with 5 engineers)

Appendix B: TCO Analysis Example

Scenario: 100TB Data, 1000 Queries/Day

Snowflake Costs (Annual): - Storage: $100\text{TB} \times \$40/\text{TB}/\text{month} \times 12 = \$48,000$ - Compute: $8 \text{ hours/day} \times \text{Large warehouse } (\$4/\text{credit}) \times 4 \text{ credits/hour} \times 365 \text{ days} = \$46,720$ - Data Transfer: $10\text{TB}/\text{month} \times \$0.08/\text{GB} \times 12 = \$9,600$ - **Total:** ~\$104,320/year

Orbit-RS (Self-Hosted on AWS): - Storage (S3): $100\text{TB} \times \$23/\text{TB}/\text{month} \times 12 = \$27,600$ - Compute (EC2): $3 \times \text{r6i.4xlarge } (\$1.008/\text{hr}) \times 24 \times 365 = \$26,530$ - Data Transfer: $10\text{TB}/\text{month} \times \$0.09/\text{GB} \times 12 = \$10,800$ - **Total:** ~\$64,930/year

Savings: \$39,390/year (38% reduction)

With Orbit-RS Managed Cloud (hypothetical): - Estimated pricing: 60% of Snowflake = ~\$62,592/year
- **Savings:** \$41,728/year (40% reduction)

Appendix C: Glossary

- **MVCC:** Multi-Version Concurrency Control
- **Time Travel:** Ability to query historical versions of data
- **Zero-Copy Clone:** Create table copy without duplicating data
- **Snowpipe:** Snowflake's continuous data ingestion service
- **Snowpark:** Snowflake's DataFrame API for Python/Scala/Java
- **Secure Data Sharing:** Share data between accounts without copying
- **TCO:** Total Cost of Ownership

Document Classification: Internal Strategic Planning **Revision History:** - v1.0 (2025-12-13): Initial comprehensive analysis

Next Review: Q1 2026 (post-Phase 1 delivery)